MBLP131 · HAFTA 4

İşletim Sistemleri Temelleri

Hafta 4 · İş Parçacıkları ve İşlemci Zamanlamasının Temelleri

Bir program aynı anda birkaç işi nasıl yapar — ve sırada kimin çalışacağına kim karar verir?

Elektronik Haberleşme Teknolojisi Programı · Meslek Yüksekokulu · İstanbul Okan Üniversitesi

GEÇEN HAFTA

Dört Fikirde 3. Hafta

Süreç

Belleği: metin (kod), veri, bss, öbek (heap) ve yığıt (stack) — ayrıca işlemci durumu ve açık dosyalar.

Beş durum

yeni → hazır → çalışıyor → bekliyor → sonlandı. Her çekirdekte aynı anda tek süreç çalışır.

PCB

Çekirdeğin süreç kaydı: durum, PC, yazmaçlar, bellek bilgisi, açık dosyalar, zamanlama bilgisi.

Bağlam değişimi

Bir sürecin PCB'si kaydedilir, diğerininki yüklenir. fork() yaratır, exec() değiştirir, wait() toplar.

Bugün: bir süreç birden çok yürütme akışı (iş parçacığı) içerebilir — ve işletim sisteminin sırada hangisinin çalışacağına dair bir kurala ihtiyacı vardır.

Bugünün Gündemi

1. **İş parçacıkları** Neden iş parçacığı? Neleri paylaşırlar, neleri kendilerine saklarlar; yararları.
2. **Çok çekirdek** Eşzamanlılık ve paralellik; çözümlü örnekle Amdahl yasası.
3. **Modeller** Kullanıcı ve çekirdek iş parçacıkları; çoktan bire, birebir, çoktan çoğa; Pthreads.
4. **Zamanlama** İşlemci–G/Ç patlamaları, hazır kuyruğu, kesintili zamanlama, dağıtıcı.
5. **Ölçütler** Kullanım, üretilen iş, dönüş, bekleme ve yanıt süresi — hesaplamalı.

BÖLÜM 1

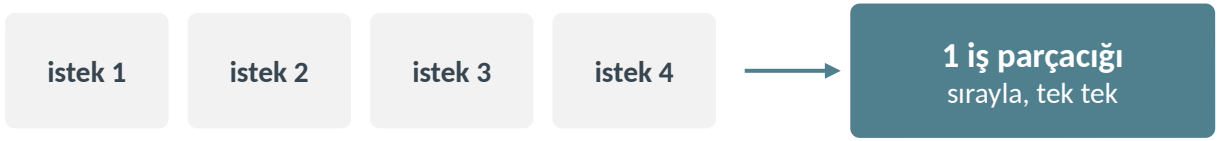
İş Parçacıkları

Tek süreç, birden çok yürütme akışı.

Neden İş Parçacığı? Bir Web Sunucusu Örneği

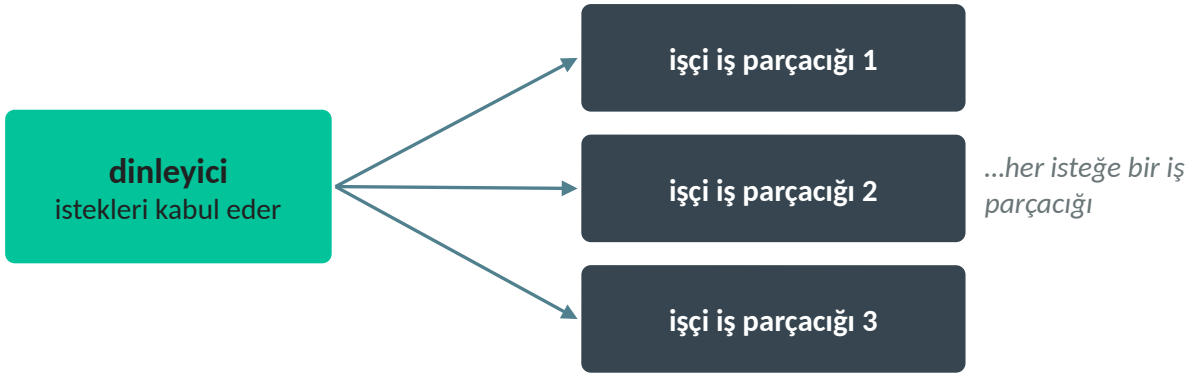
Bir web sunucusu saniyede binlerce istek alır. Her istek biraz işlemci zamanı ve çok fazla bekleme (disk, ağ) gerektirir.

Tek iş parçacıklı sunucu



1. istek diski beklerken 2–4. istekler sırada bekler.

Çok iş parçacıklı sunucu



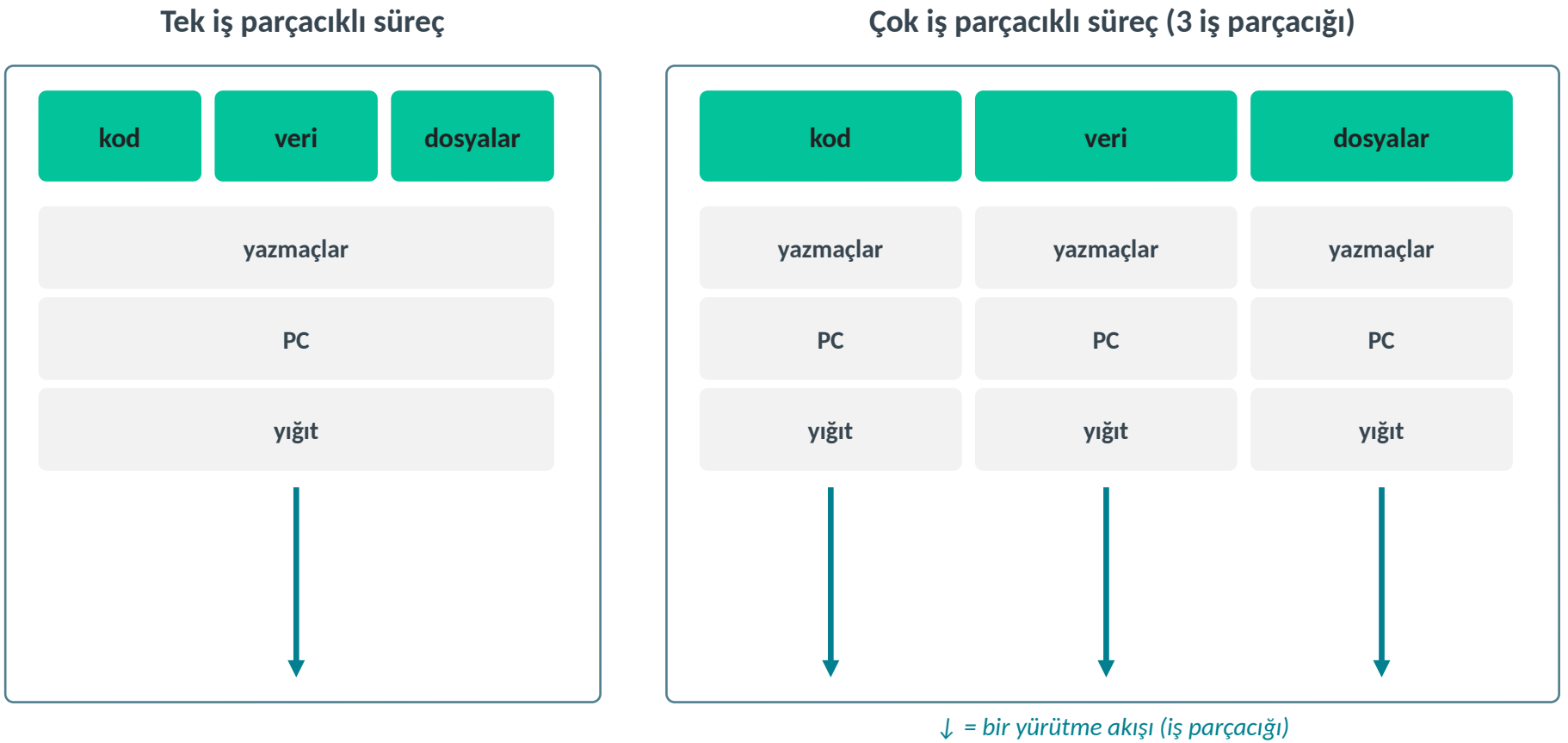
Aynı fikir her yerde

- **Kelime işlemci:** bir iş parçacığı tuşları okur, biri ekranı yeniden çizer, biri yazımı denetler, biri otomatik kaydeder.
- **Tarayıcı:** her sekme aynı anda sayfa yükler, resim çözer, betik çalıştırır.
- **Oyun:** görüntü, fizik, ses ve ağ ayrı iş parçacıklarında.
- Hepsi aynı belge / sayfa / dünya üzerinde çalışır — dolayısıyla belleği paylaşmaları gerekir.

TANIM

İş Parçacığı, İşlemcide Yürütmenin Birimidir

Her iş parçacığının kendi kimliği (TID), program sayacı (PC), yazmaç kümesi ve yığıtı vardır. Gerisini sürecindeki diğer iş parçacıklarıyla paylaşır.



Not

Geleneksel ("ağır siklet") süreç, tam olarak bir iş parçacığı olan süreçtir.

3. haftanın PCB'si artık ikiye ayrılır: sürece ait kısım + her iş parçacığı için küçük bir blok.

İş Parçacıkları Neyi Paylaşır? Ne Kendilerine Özeldir?

Öğe	Bir sürecin iş parçacıkları...	Neden
Kod (metin bölümü)	paylaşır	Tüm iş parçacıkları aynı programın fonksiyonlarını çalıştırır.
Global veri ve öbek	paylaşır	Global bir değişken ya da malloc ile ayrılmış bir dizi her iş parçacığınca görülür.
Açık dosyalar, PID, kullanıcı kimliği	paylaşır	Kaynaklar iş parçacığına değil, sürece aittir.
Program sayacı ve yazmaçlar	özel	Her iş parçacığı kodda kendi yerindedir.
Yığıt (yerel değişkenler)	özel	Her iş parçacığının kendi fonksiyon çağrı zinciri vardır.
İş parçacığı kimliği, zamanlama durumu	özel	İşletim sistemi her iş parçacığını ayrı zamanlar.

Paylaşım işin özüdür — ve tehlikesidir: aynı değişkeni aynı anda değiştiren iki iş parçacığı yanlış sonuç üretir (yarış durumu, race condition; 5. hafta).

YARARLAR

Çok İş Parçacıklı Programlamanın Dört Yararı

1. Yanıt verebilirlik

Bir iş parçacığı bloke olsa ya da uzun bir hesap yapsa bile diğerleri çalışmayı sürdürür — kullanıcı arayüzü donmaz.

2. Kaynak paylaşımı

İş parçacıkları belleği ve dosyaları kendiliğinden paylaşır. Süreçlerin iş birliği için ek mekanizmalar (paylaşılan bellek, borular) gerekir.

3. Ekonomi

İş parçacığı yaratmak ve iş parçacıkları arasında geçiş yapmak, süreçlere göre çok daha ucuzdur: yeni adres alanı gerekmez.

4. Ölçeklenebilirlik

Çok çekirdekli işlemcide bir sürecin iş parçacıkları farklı çekirdeklerde paralel çalışabilir. Tek iş parçacıklı süreç yalnızca bir çekirdek kullanır.

Sonraki slayt: 3. yararı ölçüyoruz — iş parçacığı süreçten ne kadar ucuz?

UYGULAMA

Örnek 1: İş Parçacığı Gerçekten Daha mı Ucuz?

```c
#define N 5000
void *nothing(void *arg) { return arg; }

/* 1) N süreç */
for (int i = 0; i < N; i++) {
    pid_t pid = fork();            /* süreci kopyala   */
    if (pid == 0) _exit(0);        /* çocuk çıkar      */
    waitpid(pid, NULL, 0);
}
/* 2) N iş parçacığı */
for (int i = 0; i < N; i++) {
    pthread_t t;
    pthread_create(&t, NULL, nothing, NULL);
    pthread_join(t, NULL);
}
/* programın tamamı: code/create_cost.c */
```

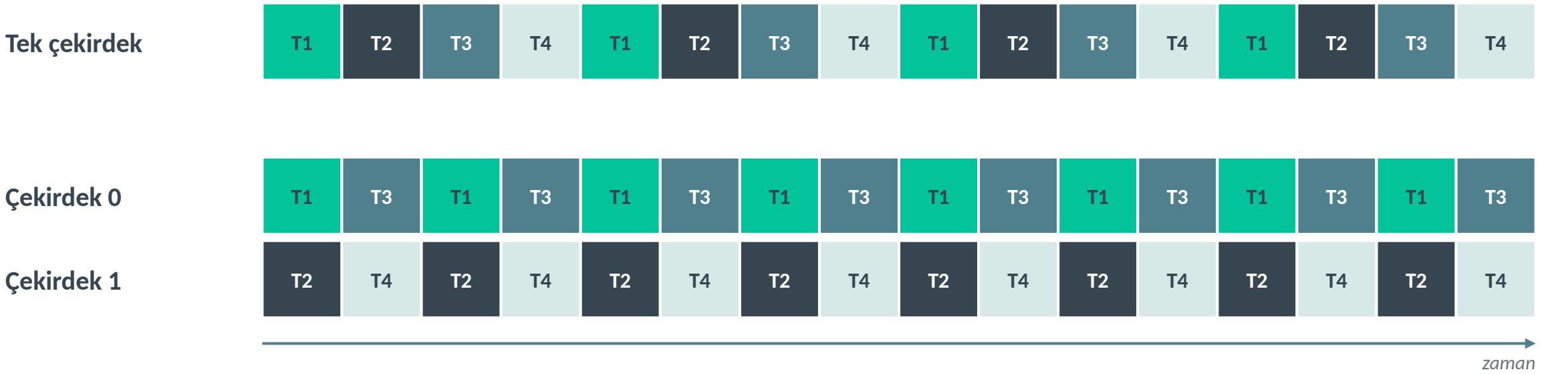
Deney

- 5000 süreç yarat (fork + wait); her biri hemen çıkar.
- Sonra hiçbir şey yapmayan 5000 iş parçacığı yarat (pthread_create + join).
- 2 çekirdekli bir Linux sanal makinesinde ölçüldü:

```
$ ./create_cost
fork + wait          :  205.5 us per process
pthread_create + join:   51.9 us per thread
a thread is about 4 times cheaper to create
```

Eşzamanlılık ve Paralellik

Zaman içinde dört iş parçacığı, T1–T4. Aynı program, iki farklı makine.



Eşzamanlılık (concurrency)

Görevler sırayla ilerler — TEK çekirdekte bile.

Paralellik (parallelism)

Görevler AYNI anda yürür — en az 2 çekirdek gerekir.

Çok Çekirdek İçin Programlama: Beş Zorluk

Görev bulma

İşin bağımsız çalışabilecek parçalarını belirlemek.

Denge

Her çekirdeğe benzer miktarda iş vermek; yoksa bazıları boşta kalır.

Veriyi bölmek

Veriyi, her iş parçacığı kendi parçasında çalışacak biçimde ayırmak.

Veri bağımlılığı

B görevi A'nın sonucuna muhtaçsa B beklemelidir: eşzamanlama.

Test

Olası iç içe geçiş sırası çok fazladır: hatalar yalnızca bazen ortaya çıkar.

Veri paralelliği: verinin farklı parçalarına aynı işlem (dizi toplama örneğimiz). Görev paralelliği: aynı anda farklı işlemler (kelime işlemcinin iş parçacıkları).

Amdahl Yasası: Çekirdek Eklemenin Sınırı

Her programın bir kısmı seri çalışır (bölünemez). Bu kısım, çekirdek sayısı arttıkça hızlanmaz.

$$\text{hızlanma} \le \frac{1}{S + (1 - S) / N}$$

S = işin seri kesri, N = çekirdek sayısı



Formülü okumak

- 1 çekirdekte program S + (1 − S) = 1 sürer.
- N çekirdekte yalnızca paralel kısım kısalır: (1 − S)/N.
- **Sınır:** sonsuz sayıda çekirdekle bile hızlanma ≤ 1/S.
- %25 seri ⇒ kaç çekirdek alırsanız alın, asla 4 kattan fazla hızlanmaz.

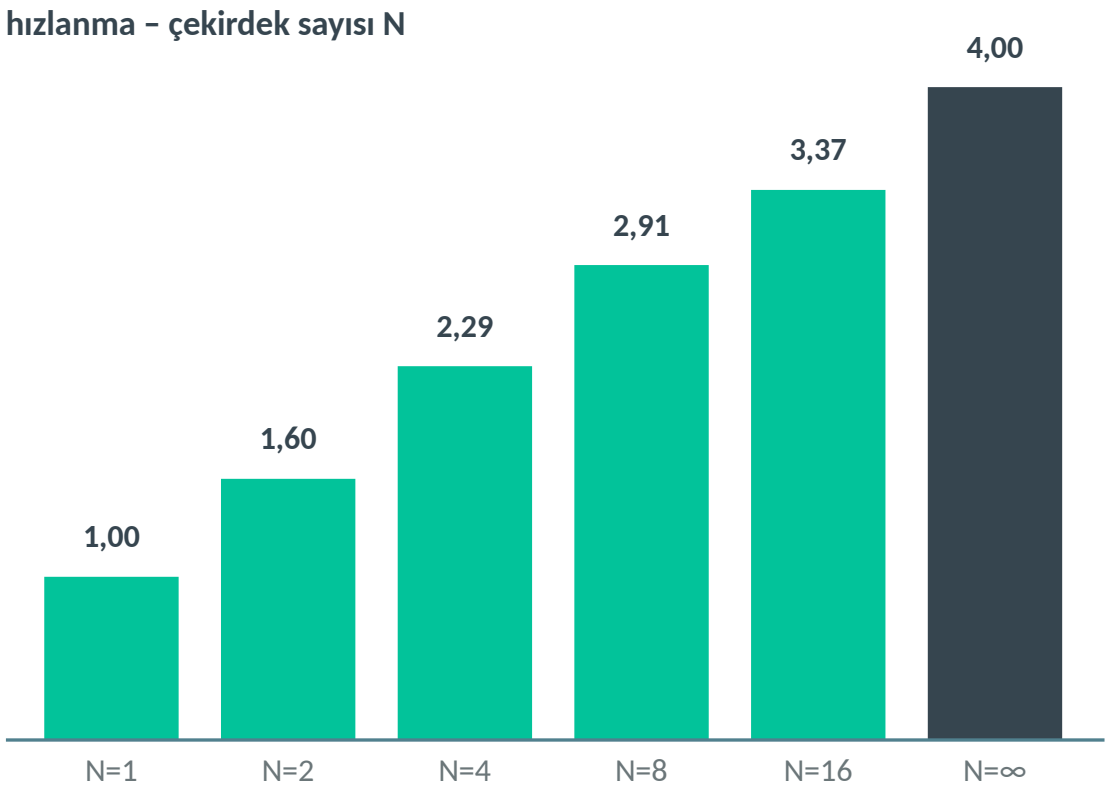
Çözümlü Örnek: %75 Paralel, %25 Seri

```
S = 0.25, yani 1 − S = 0.75

N = 2:   1 / (0.25 + 0.75/2)
       = 1 / 0.625   = 1.60

N = 4:   1 / (0.25 + 0.75/4)
       = 1 / 0.4375  ≈ 2.29

N → ∞:   1 / 0.25    = 4.00
```



8 çekirdekten 16 çekirdeğe çıkmak (donanımı ikiye katlamak) hızlanmayı yalnızca 2,91'den 3,37'ye yükseltir. Önce seri kısmı küçültün — sonra çekirdek ekleyin.

Kullanıcı Düzeyi ve Çekirdek Düzeyi İş Parçacıkları

İş parçacıklarının varlığından kim haberdar — yalnızca programın içindeki bir kütüphane mi, yoksa çekirdeğin kendisi mi?

Kullanıcı düzeyi (user-level)

- Süreç içindeki bir kütüphane yönetir; çekirdek tek iş parçacıklı tek bir süreç görür.
- Yaratma ve geçiş: yalnızca fonksiyon çağrıları — çok hızlı, sistem çağrısı yok.
- Bir iş parçacığı bloke eden bir sistem çağrısı yaparsa çekirdek bütün süreci bloke eder.
- İki çekirdeği kullanamaz: çekirdek tek bir şeyi zamanlar.

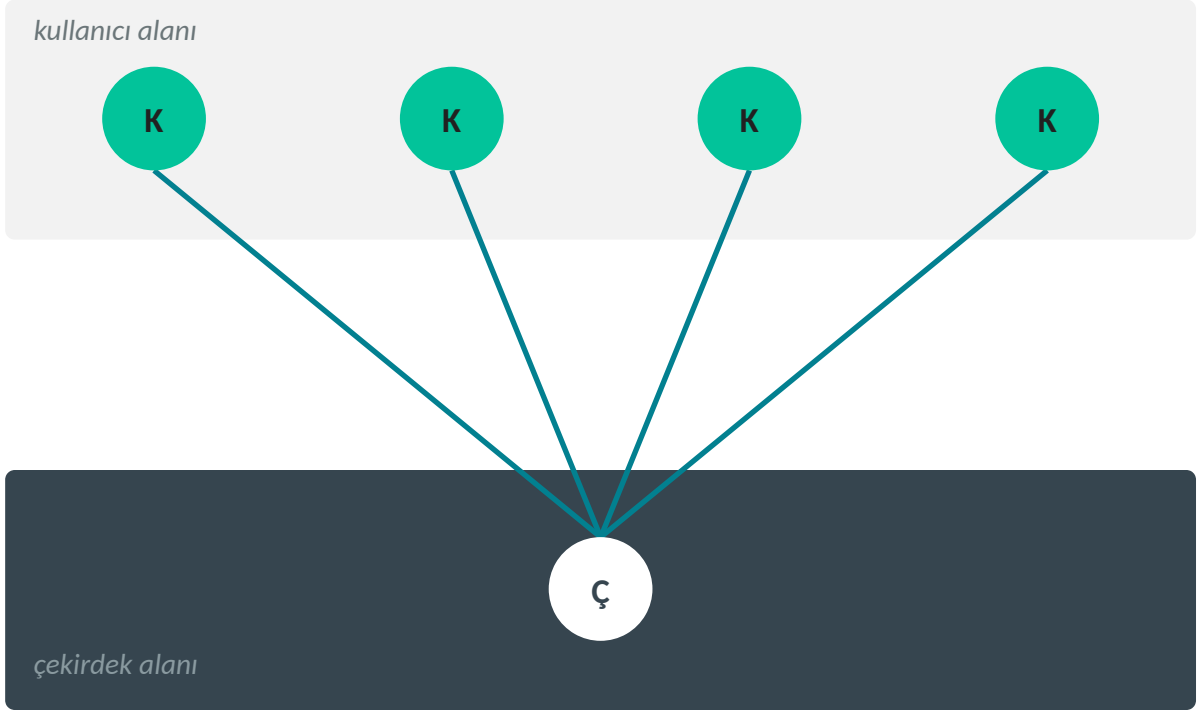
Çekirdek düzeyi (kernel-level)

- İşletim sistemi çekirdeği yaratır ve zamanlar; her birinin kendi çekirdek kaydı vardır.
- Yaratma ve geçiş sistem çağrısı ister — kullanıcı düzeyine göre yavaştır.
- Bir iş parçacığı bloke olursa çekirdek aynı sürecin başka bir iş parçacığını çalıştırır.
- Bir sürecin iş parçacıkları birkaç çekirdekte paralel çalışır.

Her kullanıcı iş parçacığı eninde sonunda bir çekirdek iş parçacığı üzerinde çalışmalıdır. Bu eşleştirmenin biçimine çoklu iş parçacığı modeli denir.

Model 1: Çoktan Bire (Many-to-One)

Birçok kullanıcı iş parçacığı tek bir çekirdek iş parçacığına eşlenir. Yönetimi kullanıcı alanındaki bir kütüphane yapar.



K = kullanıcı iş parçacığı, Ç = çekirdek iş parçacığı

Avantajlar

- Hızlı: iş parçacığı geçişi çekirdeğe uğramadan yapılır.
- Çekirdek iş parçacığı olmayan bir işletim sisteminde bile çalışır.

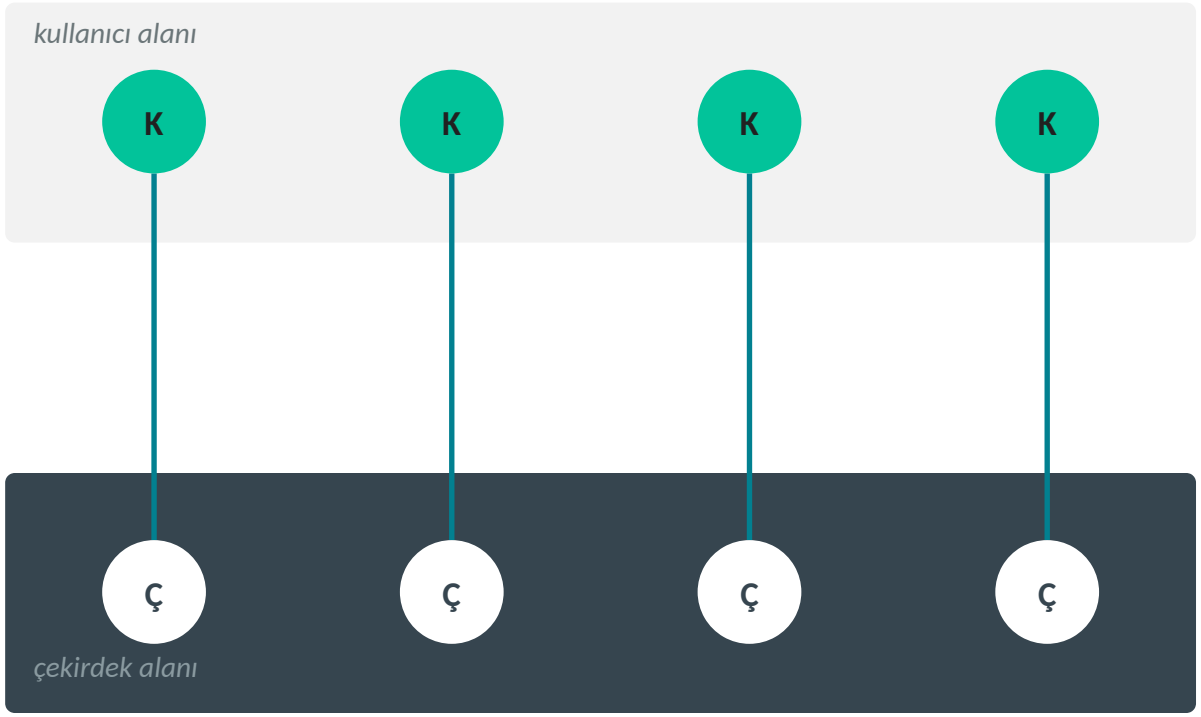
Dezavantajlar

- Bloke eden tek bir çağrı TÜM iş parçacıklarını durdurur.
- Paralellik yok: çekirdekte aynı anda tek iş parçacığı olabilir.

Örnekler: Solaris Green Threads, GNU Portable Threads. Bugün nadiren kullanılır.

Model 2: Birebir (One-to-One)

Her kullanıcı iş parçacığı kendi çekirdek iş parçacığına eşlenir. pthread_create() çekirdekten yeni bir iş parçacığı ister.



Her pthread_create(), çekirdeğin zamanladığı yeni bir iş parçacığı yaratır.

Avantajlar

- Bloke eden bir çağrı yalnızca o iş parçacığını bloke eder.
- Çok çekirdekli işlemcilerde gerçek paralellik.

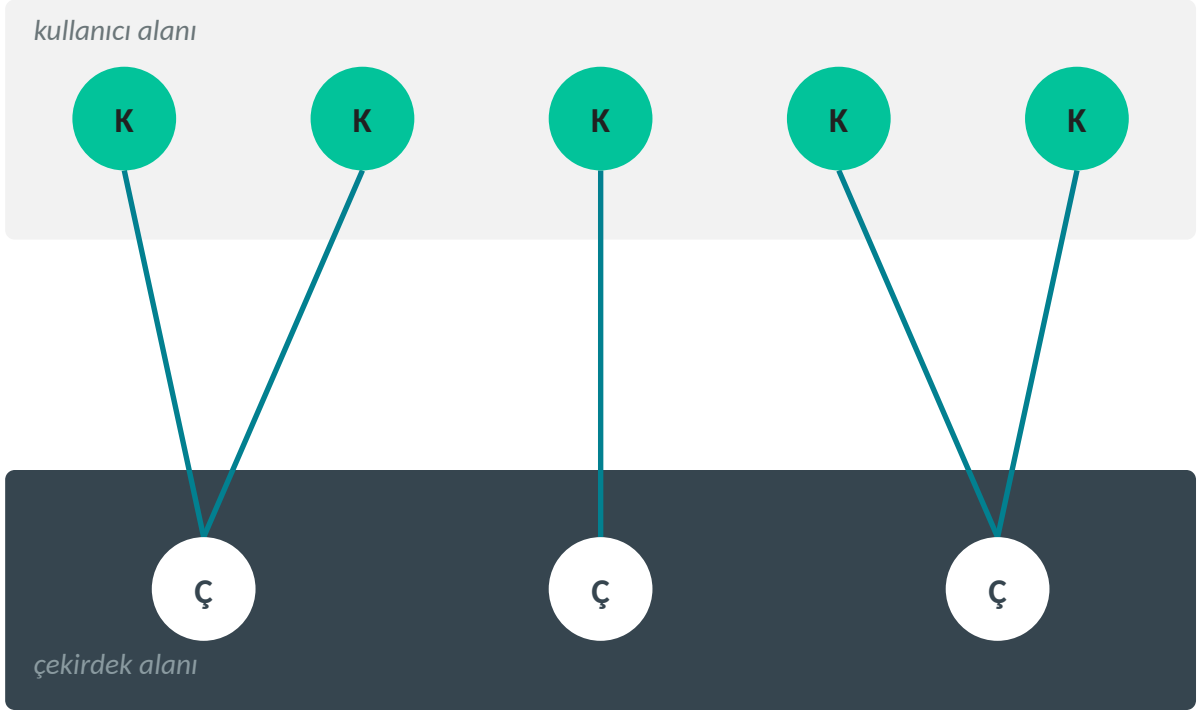
Dezavantajlar

- Her kullanıcı iş parçacığı bir çekirdek iş parçacığına mal olur: yaratmak sistem çağrısı ister.
- Çok sayıda iş parçacığı çekirdeğe yük olabilir.

Linux (NPTL) ve Windows bu modeli kullanır — bugün programlarımızda göreceğimiz model.

Model 3: Çoktan Çoğa (Many-to-Many)

Birçok kullanıcı iş parçacığı, daha az ya da eşit sayıda çekirdek iş parçacığı üzerine çoğullanır.



Hangi kullanıcı iş parçacığının hangi çekirdek iş parçacığında çalışacağına kütüphane karar verir.

Avantajlar

- İstediğiniz kadar kullanıcı iş parçacığını ucuza yaratabilirsiniz.
- Çok çekirdekte yine paraleldir; bloke eden bir çağrı her şeyi durdurmaz.

Dezavantajlar

- Gerçeklemesi karmaşıktır: iki zamanlayıcı (kütüphane + çekirdek) uyum içinde çalışmalıdır.

Türevi iki düzeyli model: bir iş parçacığını 1:1 bağlamaya da izin verir. Modern biçimi: Go goroutine'leri, Java sanal iş parçacıkları.

ÖZET

Üç Model Yan Yana

Model	Kullanıcı : çekirdek	Çok çekirdekte paralel mi?	Bloke eden çağrı durdurur…	Kullananlar
Çoktan bire	N : 1	hayır	tüm iş parçacıklarını	Green threads (eski)
Birebir	1 : 1	evet	yalnızca o iş parçacığını	Linux, Windows
Çoktan çoğa	N : M (M ≤ N)	evet	yalnızca o iş parçacığını	Go, Java sanal iş parçacıkları

Linux ve Windows birebir modeli kullanır: pthread_create() ile yarattığınız her iş parçacığı, çekirdeğin zamanlayıcısında ayrı bir varlıktır.

POSIX İş Parçacıkları (Pthreads): Temel Çağrılar

Üç ana iş parçacığı kütüphanesi: POSIX Pthreads (Linux, macOS), Windows iş parçacığı API'si ve Java iş parçacıkları.

Çağrı	Ne yapar?
pthread_create(&t, NULL, f, arg)	f(arg) fonksiyonunu çalıştıran yeni bir iş parçacığı başlatır. Tutamacı t'ye yazılır.
pthread_join(t, &result)	t iş parçacığı bitene kadar bekler (süreçlerdeki wait() gibi).
pthread_exit(value)	Çağıran iş parçacığını sonlandırır (f'den dönmek de aynı işi görür).
pthread_self()	Çağıran iş parçacığının kendi tutamacını döndürür.
gcc -pthread prog.c	Pthreads kütüphanesiyle derler ve bağlar.

Pthreads bir gerçekleme değil, bir belirtimdir (POSIX standardı). Linux'ta glibc (NPTL) tarafından clone() sistem çağrısının üzerine gerçeklenir.

UYGULAMA

Örnek 2: Üç İş Parçacığından Merhaba

```c
int shared_counter = 0;          /* global: TEK kopya */

void *worker(void *arg)
{
    int my_number = *(int *)arg; /* KENDİ yığıtımda */
    printf("thread %d: PID = %d, TID = %d\n",
           my_number, getpid(), gettid());
    shared_counter++;
    return NULL;
}

int main(void)
{
    pthread_t tid[3];  int number[3];
    for (int i = 0; i < 3; i++) {
        number[i] = i + 1;
        pthread_create(&tid[i], NULL, worker, &number[i]);
    }
    for (int i = 0; i < 3; i++)
        pthread_join(tid[i], NULL);   /* bekle */
    printf("shared_counter = %d\n", shared_counter);
}
/* programın tamamı: code/thread_hello.c */
```

Neye dikkat edelim?

- Tüm iş parçacıkları aynı PID'yi yazar: tek süreç.
- Her iş parçacığının kendi TID'si var (bir çekirdek iş parçacığı: birebir model).
- İş parçacıkları herhangi bir sırada biter: 3, 2'den önce yazdı!

```
$ ./thread_hello
main    : PID = 6351, TID = 6351
thread 1: PID = 6351, TID = 6352
thread 3: PID = 6351, TID = 6354
thread 2: PID = 6351, TID = 6353
main    : all threads done,
          shared_counter = 3
```

UYGULAMA

Örnek 3: Bir Diziyi T İş Parçacığıyla Toplamak

```
int  *a;              /* büyük dizi: ortak      */
long  partial[MAXT];  /* her iş parçacığına bir */

void *sum_slice(void *arg)
{
    long k = (long)arg;               /* dilimim */
    long start = k * N / T, end = (k + 1) * N / T;
    long s = 0;
    for (long i = start; i < end; i++)
        s += a[i];
    partial[k] = s;                   /* gözüm   */
    return NULL;
}

/* main() içinde: */
for (long k = 0; k < T; k++)
    pthread_create(&th[k], NULL, sum_slice, (void *)k);
for (long k = 0; k < T; k++) {
    pthread_join(th[k], NULL);
    total += partial[k];
}
/* tam program (zaman ölçümü, 20 tekrar): code/array_sum.c */
```

Veri paralelliği

- 100 milyon int (400 MB), T eşit dilime bölünür.
- k. iş parçacığı k. dilimi partial[k]'ya toplar — sonuçlar paylaşılmaz, kilit gerekmez.
- main() tüm iş parçacıklarını bekler (join) ve kısmi toplamları ekler.

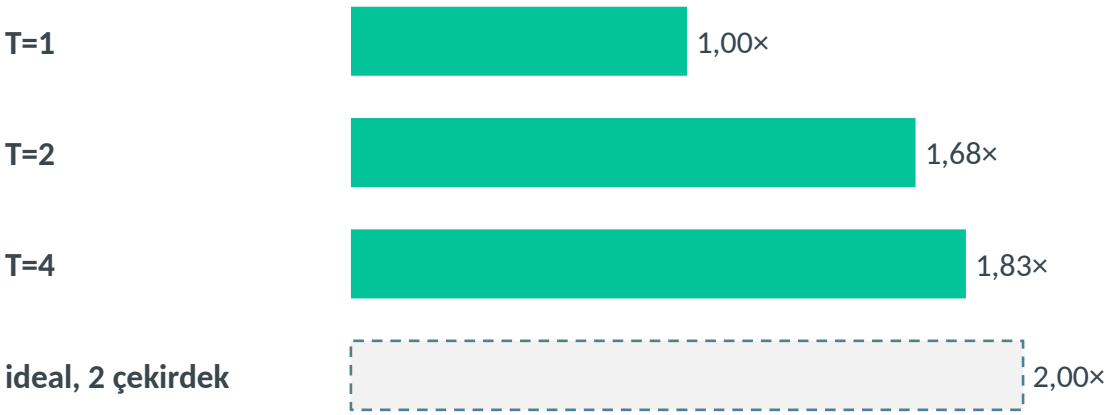
```
$ ./array_sum 1
threads = 1  sum = 9000000000  time = 1.686 s
$ ./array_sum 2
threads = 2  sum = 9000000000  time = 0.999 s
```

UYGULAMA

Ölçüm: 2 Çekirdekte 1, 2 ve 4 İş Parçacığı

Test sanal makinemizde 9 çalıştırmanın ortancası ($ nproc → 2). Makine başka işlerle paylaşıldığından tek tek çalıştırmalar ±%30'a kadar değişti.

İş parçacığı T	Süre (ortanca)	Hızlanma
1	1,564 s	1,00
2	0,932 s	1,68
4	0,855 s	1,83



Yorum

- **2 iş parçacığı:** 1,68× daha hızlı — iyi, ama ideal 2×'in altında.
- **4 iş parçacığı:** pek daha iyi değil: yalnızca 2 çekirdek var, herhangi bir anda en çok 2 iş parçacığı çalışır.
- **Neden 2× değil?** iş parçacıklarını yaratma ve bekleme, ortak bellek bant genişliği ve makinedeki diğer işler.
- Amdahl: 2 çekirdekte 1,68× sanki S ≈ 0,19 gibi davranır — "seri kısım" tüm bu ek yükleri içerir.

UYGULAMA

İş Parçacıklarını ps ve /proc ile Görmek

```
$ ./thread_hello &      # iş parçacıkları 2 s uyur
$ ps -eLf | grep thread_hello
UID    PID  PPID   LWP  C NLWP CMD
root 27241 27234 27241  0    4 ./thread_hello
root 27241 27234 27243  0    4 ./thread_hello
root 27241 27234 27244  0    4 ./thread_hello
root 27241 27234 27245  0    4 ./thread_hello

$ ls /proc/27241/task
27241  27243  27244  27245

$ grep Threads /proc/27241/status
Threads:  4
```

Çıktıyı okumak

- **-L** her iş parçacığı için bir satır gösterir.
- **PID** tüm satırlarda aynı: tek süreç.
- **LWP** (light-weight process, hafif süreç) = iş parçacığı kimliği.
- **NLWP** = iş parçacığı sayısı: main + 3 işçi = 4.
- /proc/<pid>/task/ altında her iş parçacığı için bir klasör vardır.

ARA

10 Dakika Mola

Sırada işlemci zamanlayıcısı var: sıradaki kim çalışacak ve "iyi"yi nasıl ölçeriz?

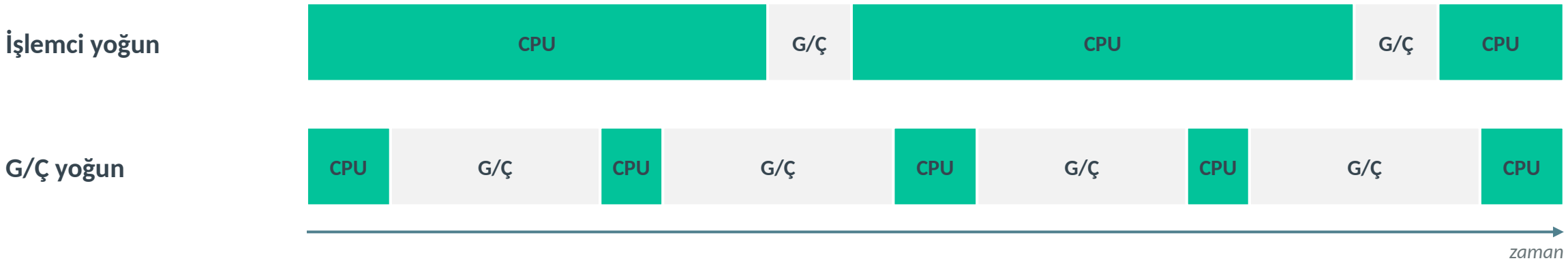
BÖLÜM 2

İşlemci Zamanlaması: Temel Kavramlar

Çok sayıda hazır iş parçacığı, az sayıda çekirdek: biri seçim yapmalı.

İşlemci–G/Ç Patlama Döngüsü

Bir süreç hesaplama (CPU patlaması, CPU burst) ile G/Ç beklemesi (G/Ç patlaması, I/O burst) arasında gidip gelir.



İşlemci yoğun (CPU-bound)

Az sayıda, uzun CPU patlaması. Örnek: video kodlama, derleme, bilimsel benzetim.

G/Ç yoğun (I/O-bound)

Çok sayıda kısa CPU patlaması, çoğunlukla bekleme. Örnek: metin düzenleyici, web sunucusu, dosya kopyalama.

Ölçülmüş gerçek

CPU patlamalarının çoğu kısadır (birkaç ms); uzun olanlar azdır. Zamanlayıcılar buna göre tasarlanır.

Örnek 4: time ile İşlemci Yoğun ve G/Ç Yoğun

```
/* ./burst cpu : tek, uzun bir CPU patlaması */
for (long i = 1; i <= 4000000000L; i++)
    x += 1.0 / i;

/* ./burst io : 1000 x (kısa CPU + G/Ç beklemesi) */
for (int i = 0; i < 1000; i++) {
    buf[0] = 'A' + i % 26;     /* CPU    */
    write(fd, buf, 4096);      /* G/Ç... */
    fsync(fd);                 /* ...diski bekle */
}
/* programın tamamı: code/burst.c */
```

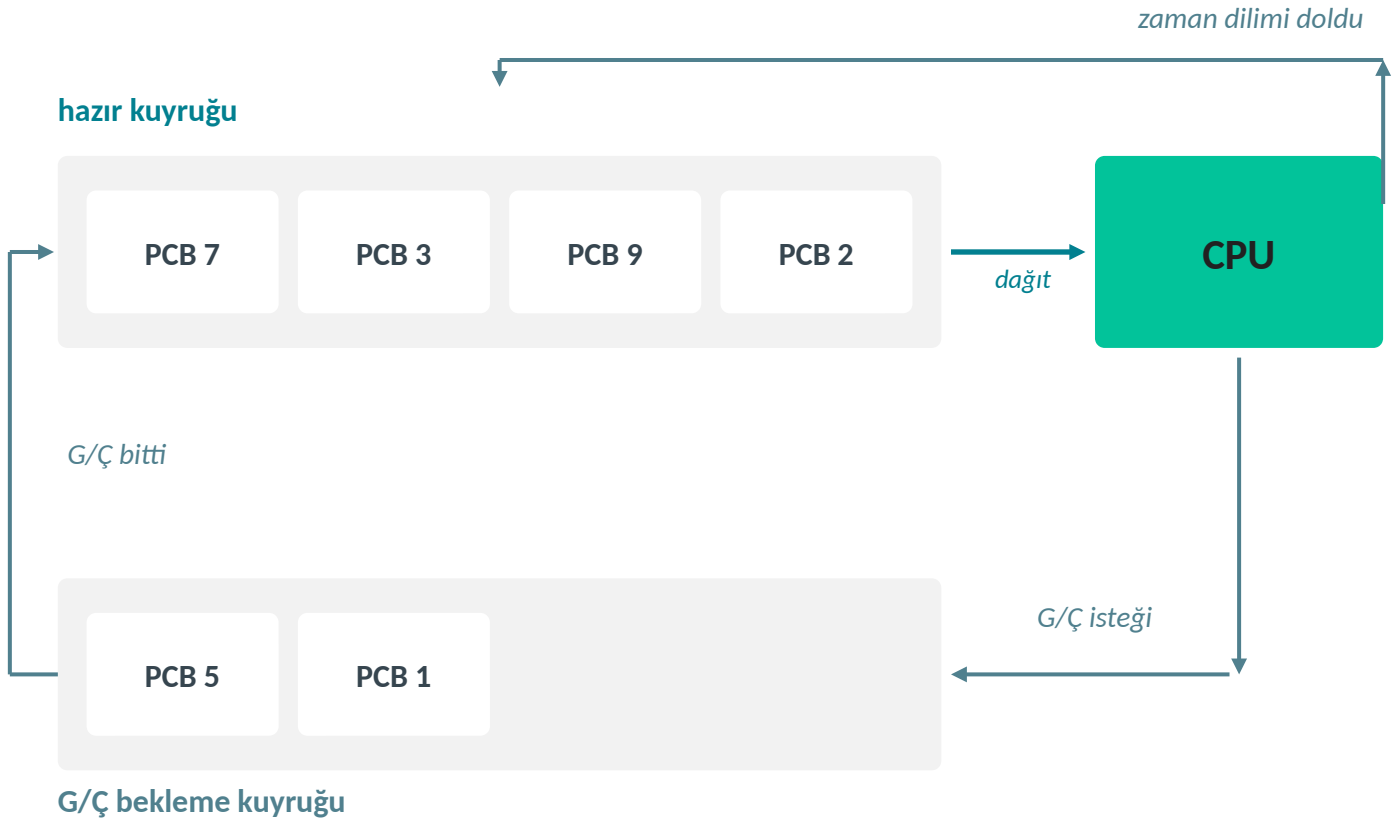
real ile user + sys

- **real:** duvar saati süresi.
- **user / sys:** kullanıcı / çekirdek modunda harcanan işlemci zamanı.
- İşlemci yoğun: real ≈ user + sys.
- G/Ç yoğun: real ≫ user + sys — sürecin zamanın yaklaşık %73'ünde beklediği, işlemcinin de başkalarına boş kaldığı anlamına gelir.

```
$ time ./burst cpu
real 0m1.266s  user 0m1.197s  sys 0m0.001s
$ time ./burst io
real 0m0.384s  user 0m0.000s  sys 0m0.104s
```

İşlemci Zamanlayıcısı ve Hazır Kuyruğu

İşlemci her boşa çıktığında kısa vadeli zamanlayıcı (short-term scheduler) hazır kuyruğundan bir süreç seçer.

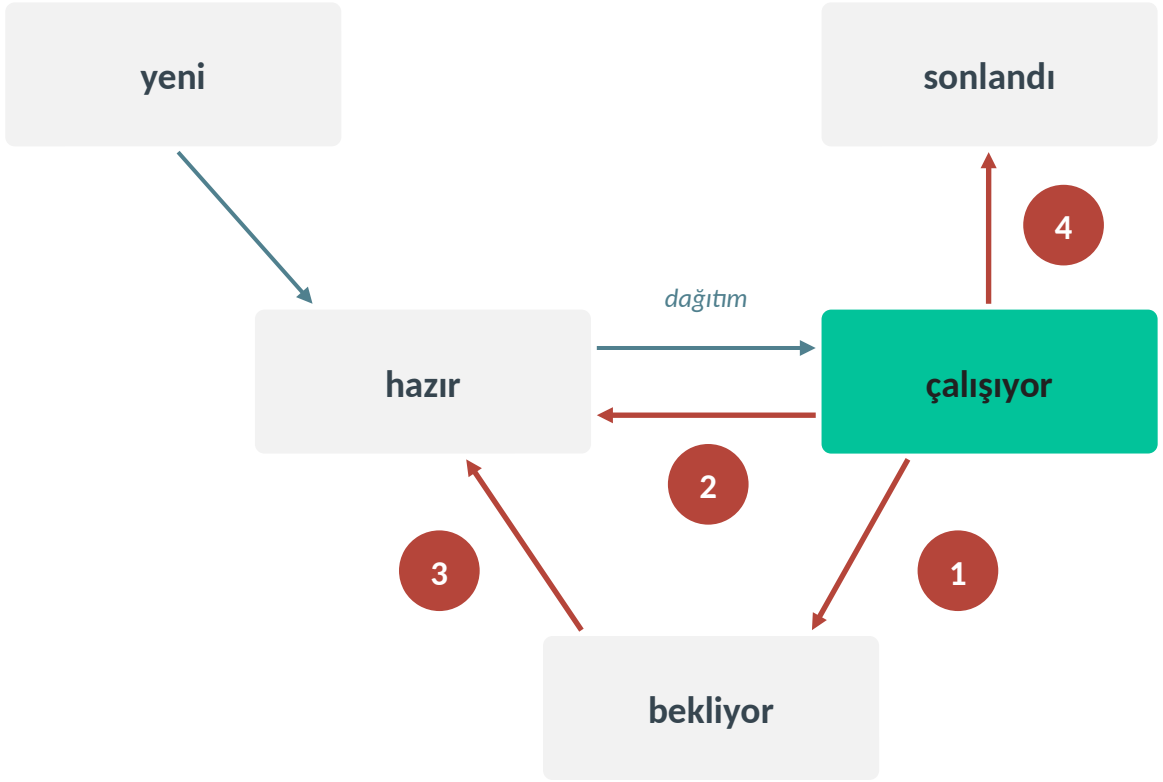


Önemli noktalar

- Hazır kuyruğu (ready queue), şu anda çalışabilecek süreçlerin PCB'lerini tutar.
- Mutlaka FIFO değildir: bir öncelik kuyruğu, bir ağaç ya da bir liste olabilir. Sırayı belirlemek zamanlama algoritmasının işidir (5. hafta).
- Kısa vadeli zamanlayıcı çok sık (birkaç ms'de bir) çalışır; bu yüzden hızlı olmalıdır.

Zamanlayıcı Ne Zaman Karar Vermek Zorunda?

Süreç durum diyagramındaki dört durum bir zamanlama kararını tetikleyebilir.



Dört durum

- **1. çalışıyor → bekliyor:** G/Ç isteği ya da bir çocuk için wait().
- **2. çalışıyor → hazır:** bir kesme; örneğin zamanlayıcı saati zaman diliminin bittiğini bildirir.
- **3. bekliyor → hazır:** G/Ç tamamlandı.
- **4. çalışıyor → sonlandı:** süreç çıkar.
- 1 ve 4'te işlemci boşalmıştır: yeni bir süreç seçilMELİDİR. 2 ve 3'te işletim sistemi geçiş yapmayı SEÇEBİLİR.

Kesintisiz ve Kesintili Zamanlama

Kesintisiz (non-preemptive, iş birlikçi)

- Zamanlama yalnızca 1. ve 4. durumlarda yapılır.
- Bir süreç işlemciyi aldığında, bloke olana ya da sonlanana kadar bırakmaz.
- Basittir, ama tek bir uzun süreç herkesi bekletebilir.
- Windows 3.x, klasik Mac OS (OS X öncesi).

Kesintili (preemptive)

- Zamanlama 2. ve 3. durumlarda da yapılır.
- İşletim sistemi işlemciyi (zamanlayıcı kesmesiyle) geri alabilir.
- Daha iyi yanıt verebilirlik; kontrolden çıkan bir program sistemi donduramaz.
- Tüm modern işletim sistemleri: Windows, macOS, Linux, Android, iOS.

Kesintinin bedeli: bir iş parçacığı ortak veriyi güncellemenin tam ortasında kesilebilir → yarış durumları. Bu, 5. haftanın ikinci yarısının konusudur.

Dağıtıcı ve Dağıtım Gecikmesi

Zamanlayıcı sırada KİMİN çalışacağına karar verir; dağıtıcı (dispatcher) işlemciyi o sürece fiilen verir.

Dağıtıcının üç görevi

- Bağlam değişimi: eski sürecin durumunu PCB'sine kaydet, yenisininkini yükle.
- Kullanıcı moduna geç.
- Yeni programda doğru yere (kaydedilmiş PC'sine) atla.



dağıtım gecikmesi

P0 → PCB0 kaydet, PCB1 → P1 yükle

Dağıtım gecikmesi saf ek yüktür: bu sırada yararlı hiçbir iş yapılmaz.

Küçüktür (mikrosaniyeler) ama çok sık yaşanır: vmstat 1 komutunu izleyin — boştaki test sanal makinemizde cs sütunu saniyede yaklaşık 500–600 bağlam değişimi gösterdi; meşgul bir sunucu on binlerce gösterir.

UYGULAMA

Bağlam Değişimlerini Saymak: Gönüllü mü, Zorla mı?

GNU time -v, sürecin işlemciyi kaç kez kendisi bıraktığını (bloke oldu) ve kaç kez elinden alındığını raporlar.

```
$ /usr/bin/time -v ./burst cpu
  User time (seconds): 1.17
  System time (seconds): 0.01
  Elapsed (wall clock) time: 0:01.22
  Voluntary context switches: 1
  Involuntary context switches: 50

$ /usr/bin/time -v ./burst io
  User time (seconds): 0.00
  System time (seconds): 0.11
  Elapsed (wall clock) time: 0:00.46
  Voluntary context switches: 3714
  Involuntary context switches: 0
```

Sonucu okumak

- **Voluntary (gönüllü):** süreç bloke oldu (1. durum) — G/Ç yoğun programlara özgü.
- **Involuntary (zorla):** zamanlayıcı işlemciyi elinden aldı (2. durum) — işlemci yoğun programlara özgü.
- 1,2 s'de 50 kesinti: işlemci yoğun program ortalama yaklaşık 24 ms'de bir kesildi.

Bir Zamanlayıcıyı Nasıl Değerlendiririz? Beş Ölçüt

Ölçüt	Tanım	Hedef
İşlemci kullanımı (CPU utilisation)	İşlemcinin yararlı işle meşgul olduğu zamanın oranı (hafif yükte %40 … ağır yükte %90).	en büyük
Üretilen iş (throughput)	Birim zamanda tamamlanan süreç sayısı.	en büyük
Dönüş süresi (turnaround)	Gelişten (sunuluştan) tamamlanmaya kadar her şey: bekleme ve G/Ç dahil.	en küçük
Bekleme süresi (waiting)	Hazır kuyruğunda geçen toplam süre. Zamanlayıcının doğrudan denetlediği tek kısım.	en küçük
Yanıt süresi (response)	Gelişten, sürecin işlemciyi ilk kez aldığı ana (ilk yanıta) kadar.	en küçük

Genellikle ortalamayı iyileştiririz. Etkileşimli sistemlerde öngörülebilir (düşük varyanslı) bir yanıt süresi, düşük bir ortalamadan daha önemli olabilir.

Tanımların Formülleri

Tek CPU patlaması olan (G/Ç yapmayan) bir süreç için; tüm zamanlar aynı saatle ölçülür.

$$T_{\text{dönüş}} = T_{\text{tamamlanma}} - T_{\text{geliş}}$$

$$T_{\text{bekleme}} = T_{\text{dönüş}} - T_{\text{patlama}}$$

$$T_{\text{yanıt}} = T_{\text{ilk çalışma}} - T_{\text{geliş}}$$

patlama = 3 + 2 = 5, bekleme = 4 + 2 = 6, dönüş = 11



Şekille sağlayalım

- Dönüş = 11 − 0 = 11.
- Bekleme = 11 − 5 = 6 (= hazır kuyruğunda 4 + 2).
- Yanıt = 4 − 0 = 4.
- G/Ç patlamaları varsa bekleme süresi yalnızca HAZIR kuyruğundaki süredir; G/Ç süresini de çıkarın.

ÇÖZÜMLÜ ÖRNEK

Alıştırma: Geliş Sırasına Göre Dört Süreç (FCFS)

"Akla ilk gelen" sıra — ilk gelen ilk hizmet alır (first come, first served) — tek işlemci, kesinti yok. Süreler ms.

Süreç	Geliş	Patlama
P1	0	5
P2	1	3
P3	2	6
P4	16	2

Asıl soruyu 5. hafta soracak: geliş sırası iyi bir sıra mı? (P2 ile P3'ün yerini değiştirip karşılaştırın.)

Gantt şeması



Nasıl okunur?

- P1 0'da gelir ve hemen çalışır. P2 ile P3, P1 çalışırken gelir: hazır kuyruğunda beklerler.
- 14'te hazır kuyruğu boştur: P4 16'da gelene kadar işlemci boşta kalır.
- Sonraki slayt: bu şemadan her ölçütü hesaplayacağız.

ÇÖZÜMLÜ ÖRNEK

Alıştırma: Ölçütleri Hesaplamak

Süreç	Geliş	Patlama	Tamamlanma	Dönüş = Tam. − Gel.	Bekleme = Dön. − Pat.	Yanıt
P1	0	5	5	5 − 0 = 5	5 − 5 = 0	0
P2	1	3	8	8 − 1 = 7	7 − 3 = 4	4
P3	2	6	14	14 − 2 = 12	12 − 6 = 6	6
P4	16	2	18	18 − 16 = 2	2 − 2 = 0	0
Ortalama				26 / 4 = 6,5 ms	10 / 4 = 2,5 ms	2,5 ms

İşlemci kullanımı = 16 ms meşgul / 18 ms = %88,9. Üretilen iş = 4 süreç / 18 ms ≈ ms başına 0,22 (saniyede 222).

Hangi Ölçüt Daha Önemli? Duruma Bağlı

İşlemci kullanımını ve üretilen işi en büyük; dönüş, bekleme ve yanıt süresini en küçük yap. Bu hedefler çatışır.

Toplu iş / sunucular

- Üretilen iş ve işlemci kullanımı.
- Saatte yapılan toplam iş yüksek olduğu sürece uzun işler sorun değildir.

Masaüstü / telefon

- Yanıt süresi: kullanıcı tepkiyi hemen görmeli.
- Öngörülebilirlik: hep 50 ms, bazen 2 s'ye çıkan 5 ms'den daha iyi hissettirir.

Gerçek zamanlı

- Son tarihlere uymak her ortalamadan önemlidir.
- Geç = yanlış (frenler, insansız hava araçları, tıbbi cihazlar).

Ödünleşim örneği: çok sık geçiş yapmak yanıt süresini iyileştirir ama dağıtım ek yükünü artırır ve üretilen işi düşürür.

KENDİNİZ DENEYİN

Kendi Makinenizde İş Parçacıkları ve Zamanlama

```
# Linux (laboratuvar, WSL ya da sanal makine)
$ nproc                    # kaç çekirdek?
$ make                     # code/ klasöründe
$ ./array_sum 1; ./array_sum 2; ./array_sum 4
$ ./thread_hello & ps -eLf | grep thread_hello
$ time ./burst cpu ; time ./burst io
$ vmstat 1 5               # cs = geçiş/s

# Windows
# Görev Yöneticisi > Ayrıntılar > "İş parçacıkları"
# Görev Yöneticisi > Performans > CPU
```

Yanıtlanacak sorular

- Makinenizde kaç çekirdek var? 2, 4 ve 8 iş parçacığıyla hangi hızlanmayı elde ediyorsunuz?
- 2 iş parçacığıyla elde ettiğiniz hızlanmadan Amdahl yasasıyla S'yi tahmin edin.
- Bilgisayarınızda en çok iş parçacığına sahip program hangisi?
- Bir sayfayı okurken tarayıcınız işlemci yoğun mu, G/Ç yoğun mu? Ya video oynatırken?

ÖZET

4. Hafta — Temel Çıkarımlar

- **İş parçacığı:** kendi PC'si, yazmaçları ve yığıtı olan bir yürütme birimi; bir sürecin iş parçacıkları kodu, veriyi, öbeği ve dosyaları paylaşır.
- **Yararlar:** yanıt verebilirlik, kaynak paylaşımı, ekonomi (burada yaratması ≈ 4× ucuz), çok çekirdekte ölçeklenebilirlik.
- **Amdahl yasası:** hızlanma ≤ 1 / (S + (1 − S)/N); seri kısım S, 1/S sınırını koyar.
- **Modeller:** çoktan bire, birebir (Linux, Windows), çoktan çoğa; Pthreads: pthread_create / pthread_join.
- **Zamanlamanın temelleri:** işlemci–G/Ç patlamaları, hazır kuyruğu, dört karar noktası, kesintili ve kesintisiz zamanlama, dağıtım gecikmesi.
- **Ölçütler:** kullanım ve üretilen işi en büyük; dönüş (tamamlanma − geliş), bekleme (dönüş − patlama) ve yanıt süresini en küçük yap.

KISA SINAV

Kendinizi Sınayın

1. Aynı sürecin iki iş parçacığı: paylaştıkları iki şeyi ve her birinin kendine özel iki şeyini sayın.

2. Bir program %90 paraleldir. 4 çekirdekte en büyük hızlanma nedir? Sonsuz sayıda çekirdekte?

3. Çoktan bire bir iş parçacığı kütüphanesi neden ikinci bir işlemci çekirdeğini kullanamaz?

4. P1 (geliş 0, patlama 4) ve P2 (geliş 1, patlama 2) FCFS ile çalışıyor. Her birinin dönüş ve bekleme süresini bulun.

GELECEK HAFTA

Zamanlama Algoritmaları ve Eşzamanlama

5. haftanın konuları:

- Zamanlama algoritmaları: FCFS, SJF / SRTF, Öncelik, Round Robin
- Her algoritma için Gantt şemaları ve ortalama bekleme süreleri
- Yarış durumları ve kritik bölge problemi
- Muteks kilitleri ve semaforlar

SORULAR?

Tartışma

Dersten sonra sorularınız olursa dersin Telegram kanalında sorun — yanıttan herkes yararlanır.

Dr. Öğr. Üyesi Hasan Oğuz